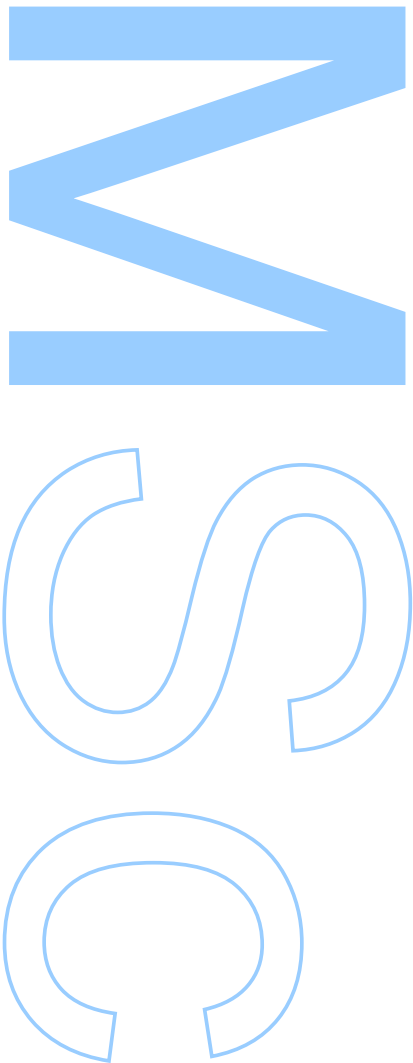
Proving Haskelite's semantics equivalent

Leonardo Santos
MSc. Computer Science
Departamento de Ciência de Computadores
2026

Orientador
João Barbosa, Faculdade de Ciências

Coorientador
Pedro Vasconcelos, Faculdade de Ciências



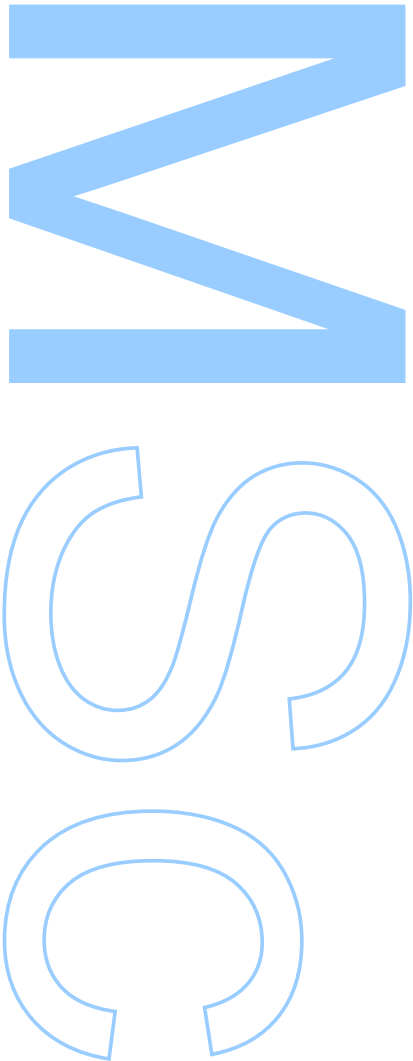
U.PORTO

FACULDADE DE CIÊNCIAS
UNIVERSIDADE DO PORTO

Todas as correções determinadas pelo júri, e só essas, foram efetuadas.

O Presidente do Júri,

Porto, _____/_____/_____



UNIVERSIDADE DO PORTO

MASTERS THESIS

Proving Haskelite's semantics equivalent

Author:
Leonardo SANTOS

Supervisor:
João BARBOSA

Co-supervisor:
Pedro VASCONCELOS

A thesis submitted in fulfilment of the requirements
for the degree of MSc. Computer Science

at the

Faculdade de Ciências da Universidade do Porto
Departamento de Ciência de Computadores

October 26, 2026

" I am and always will be the optimist, the hoper of far-flung hopes and the dreamer of improbable dreams "

Matt Smith as *The Doctor*, written by Matthew Graham

Acknowledgements

Acknowledge ALL the people!

UNIVERSIDADE DO PORTO

Abstract

Faculdade de Ciências da Universidade do Porto
Departamento de Ciência de Computadores

MSc. Computer Science

Proving Haskelite's semantics equivalent

by Leonardo SANTOS

This thesis is about something, I guess.

UNIVERSIDADE DO PORTO

Resumo

Faculdade de Ciências da Universidade do Porto
Departamento de Ciência de Computadores

Mestrado Integrado em Engenharia Física

Titulo da Tese em Portugês

por Leonardo SANTOS

Este tese é sobre alguma coisa

Contents

Acknowledgements v

Abstract vii

Resumo ix

Contents xi

List of Figures xiii

1 Introduction 1
- 1.1 Haskelite and the correspondence question . . . 1
- 1.2 Contributions and scope . . . 1

2 The language and its semantics 3
- 2.1 Expressions, patterns, and matchings . . . 3
- 2.2 Arity and weak-head normal forms . . . 3
- 2.3 Big-step evaluation . . . 3
- 2.4 The small-step machine . . . 3

3 Formalizing the semantics in Coq 5
- 3.1 Representing syntax, heaps, and judgments . . . 5
- 3.2 Binding, substitution, and local definitions . . . 5
- 3.3 Choosing fresh names . . . 5
  - 3.3.1 Canonical names and the counter-based approach . . . 5
  - 3.3.2 A shared fresh-name generator . . . 5
- 3.4 Determinism . . . 5

4 Balanced evaluations 7
- 4.1 Preserving the surrounding stack . . . 7
- 4.2 Black-holing and update frames . . . 7
- 4.3 The difficulty with linear traces . . . 7
- 4.4 Balanced evaluation trees . . . 7

5 Proving the correspondence 9
- 5.1 The theorem and its assumptions . . . 9
- 5.2 From big-step derivations to balanced trees . . . 9

5.3 From balanced trees to big-step derivations 9
5.3.1 Variable lookup and heap update 9
5.3.2 Matching results and alternatives 9
5.4 From balanced trees to machine executions 9
5.5 The remaining trace-to-tree direction 9

6 Related work **11**
6.1 Lazy semantics and machine correspondence 11
6.2 Mechanizing binding and fresh names 11

7 Conclusion **13**

A Correspondence between the paper and Coq **15**
A.1 Definitions and rules 15
A.2 Theorem statements and proof dependencies 15

Bibliography **17**

List of Figures

Chapter 1

Introduction

1.1 Haskelite and the correspondence question

1.2 Contributions and scope

Chapter 2

The language and its semantics

2.1 Expressions, patterns, and matchings

2.2 Arity and weak-head normal forms

2.3 Big-step evaluation

2.4 The small-step machine

Chapter 3

Formalizing the semantics in Coq

3.1 Representing syntax, heaps, and judgments

3.2 Binding, substitution, and local definitions

3.3 Choosing fresh names

3.3.1 Canonical names and the counter-based approach

3.3.2 A shared fresh-name generator

3.4 Determinism

Chapter 4

Balanced evaluations

4.1 Preserving the surrounding stack

4.2 Black-holing and update frames

4.3 The difficulty with linear traces

4.4 Balanced evaluation trees

Chapter 5

Proving the correspondence

5.1 The theorem and its assumptions

5.2 From big-step derivations to balanced trees

5.3 From balanced trees to big-step derivations

5.3.1 Variable lookup and heap update

5.3.2 Matching results and alternatives

5.4 From balanced trees to machine executions

5.5 The remaining trace-to-tree direction

Chapter 6

Related work

6.1 Lazy semantics and machine correspondence

6.2 Mechanizing binding and fresh names

Chapter 7

Conclusion

Appendix A

Correspondence between the paper and Coq

A.1 Definitions and rules

A.2 Theorem statements and proof dependencies

Bibliography